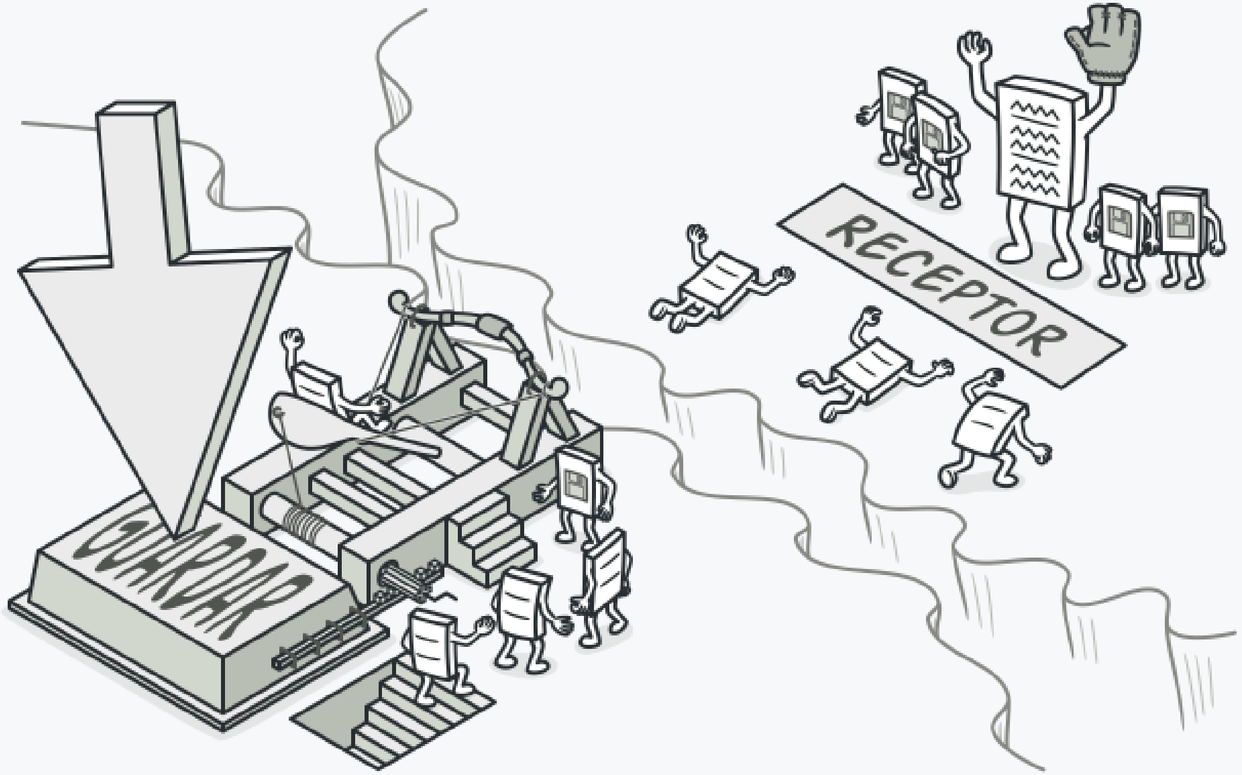
Patrón de Diseño de Comportamiento

Command

Encapsular una solicitud como un objeto.



Universidad Popular del Cesar · Ingeniería de Sistemas

Asignatura: Patrones de Diseño de Software
Integrantes: Juan Assia · Hernando Díaz · Juan Guzmán · David Palomino
Docente: Jairo Seoanes · Noviembre de 2026

¿Qué es el patrón Command?

Command es un patrón de comportamiento que convierte una solicitud en un objeto. Separa a quien solicita una acción de quien sabe ejecutarla y permite tratar las operaciones de manera uniforme.

01 · CATEGORÍA

Comportamiento

Organiza la colaboración y las responsabilidades entre objetos.

02 · PROPÓSITO

Desacoplamiento

Separa la invocación de una solicitud de su implementación.

03 · MECANISMO

Objeto comando

Encapsula una acción, su receptor y los datos necesarios.

04 · CONCEPTO CENTRAL

Solicitud como objeto

Permite configurar acciones, almacenarlas o ejecutarlas después.

PROBLEMA Y MOTIVACIÓN

¿Por qué existe Command?

El acoplamiento directo dificulta ampliar y reutilizar las operaciones.

ANTES · ACOPLAMIENTO DIRECTO

Cliente → receptor

El cliente conoce las clases y operaciones concretas y mezcla responsabilidades. Esto dificulta reutilizar, añadir acciones y gestionar colas e historial.

DESPUÉS · CON COMMAND

Cliente → comando → receptor

El emisor conoce la interfaz común y execute(), no los detalles del receptor. El comando encapsula la solicitud y facilita colas, ejecución diferida y trazabilidad.

La llamada directa es suficiente cuando no se necesita variar, almacenar o gestionar acciones.

CRITERIOS DE APLICACIÓN

¿Cuándo utilizar Command?

Cuando conviene desacoplar la solicitud de su ejecución.

Parametrizar: asociar comandos intercambiables a botones, atajos o disparadores, desacoplando emisor y receptor.

Encolar y programar: almacenar solicitudes para ejecutarlas después, incluso en segundo plano.

Historial y deshacer: registrar comandos; revertir solo cuando exista una reversión viable.



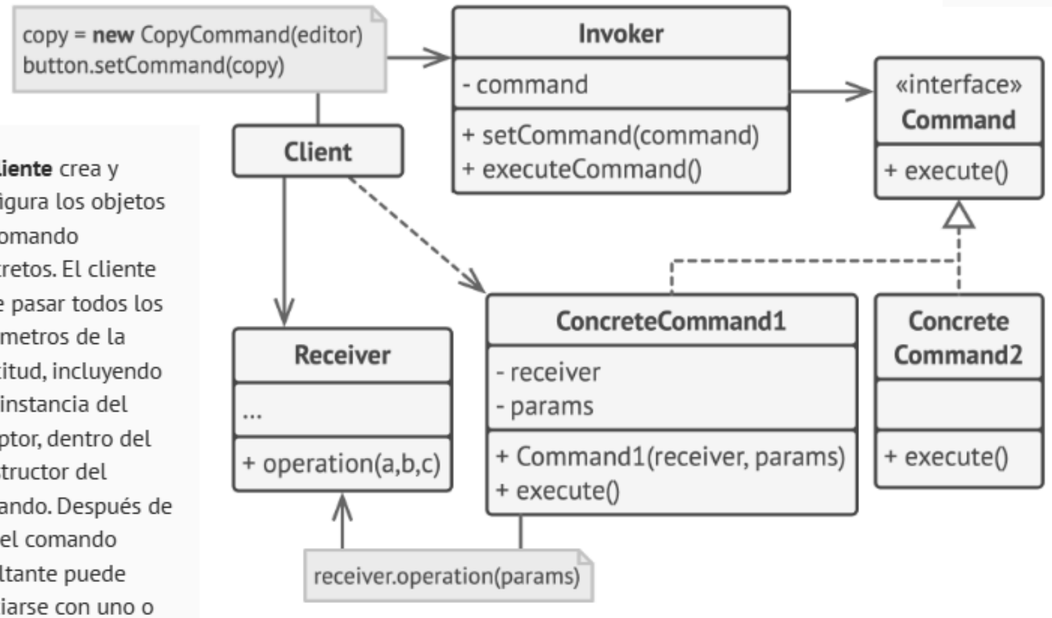
Automatización, editores, pedidos, tareas en cola, transacciones y gestión empresarial.

ESTRUCTURA ESTÁNDAR

Arquitectura del patrón Command

1 La clase **Emisora** (o *invocadora*) es responsable de inicializar las solicitudes. Esta clase debe tener un campo para almacenar una referencia a un objeto de comando. El emisor activa este comando en lugar de enviar la solicitud directamente al receptor. Ten en cuenta que el emisor no es responsable de crear el objeto de comando. Normalmente, obtiene un comando precreado de parte del cliente a través del constructor.

2 La interfaz **Comando** normalmente declara un único método para ejecutar el comando.



3 Los **Comandos Concretos** implementan varios tipos de solicitudes. Un comando concreto no se supone que tenga que realizar el trabajo por su cuenta, sino pasar la llamada a uno de los objetos de la lógica de negocio. Sin embargo, para lograr simplificar el código, estas clases se pueden fusionar.

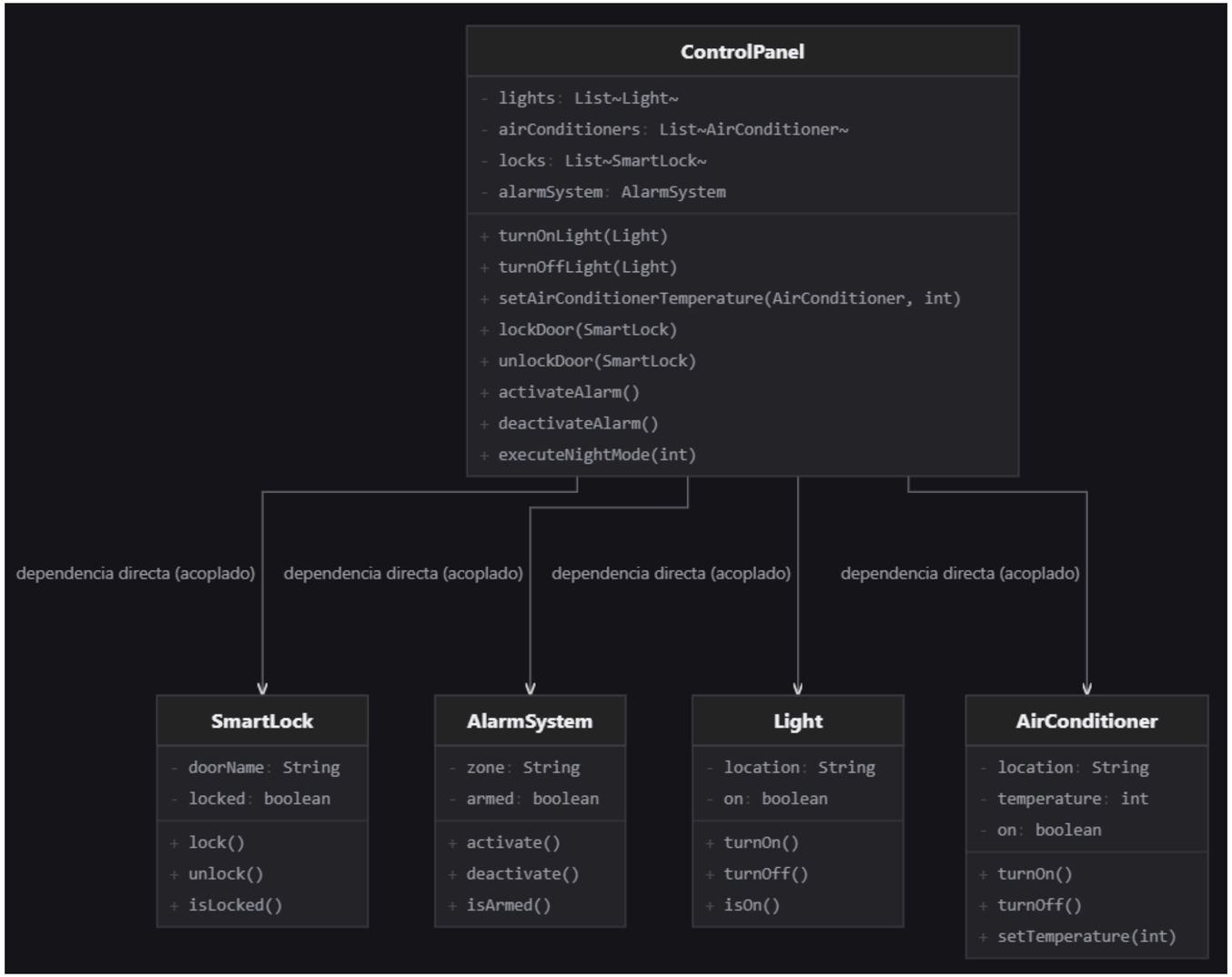
Los parámetros necesarios para ejecutar un método en un objeto receptor pueden declararse como campos en el comando concreto. Puedes hacer inmutables los objetos de comando permitiendo la inicialización de estos campos únicamente a través del constructor.

4 La clase **Receptora** contiene cierta lógica de negocio. Casi cualquier objeto puede actuar como receptor. La mayoría de los comandos solo gestiona los detalles sobre cómo se pasa una solicitud al receptor, mientras que el propio receptor hace el trabajo real.

5 El **Cliente** crea y configura los objetos de comando concretos. El cliente debe pasar todos los parámetros de la solicitud, incluyendo una instancia del receptor, dentro del constructor del comando. Después de eso, el comando resultante puede asociarse con uno o varios emisores.

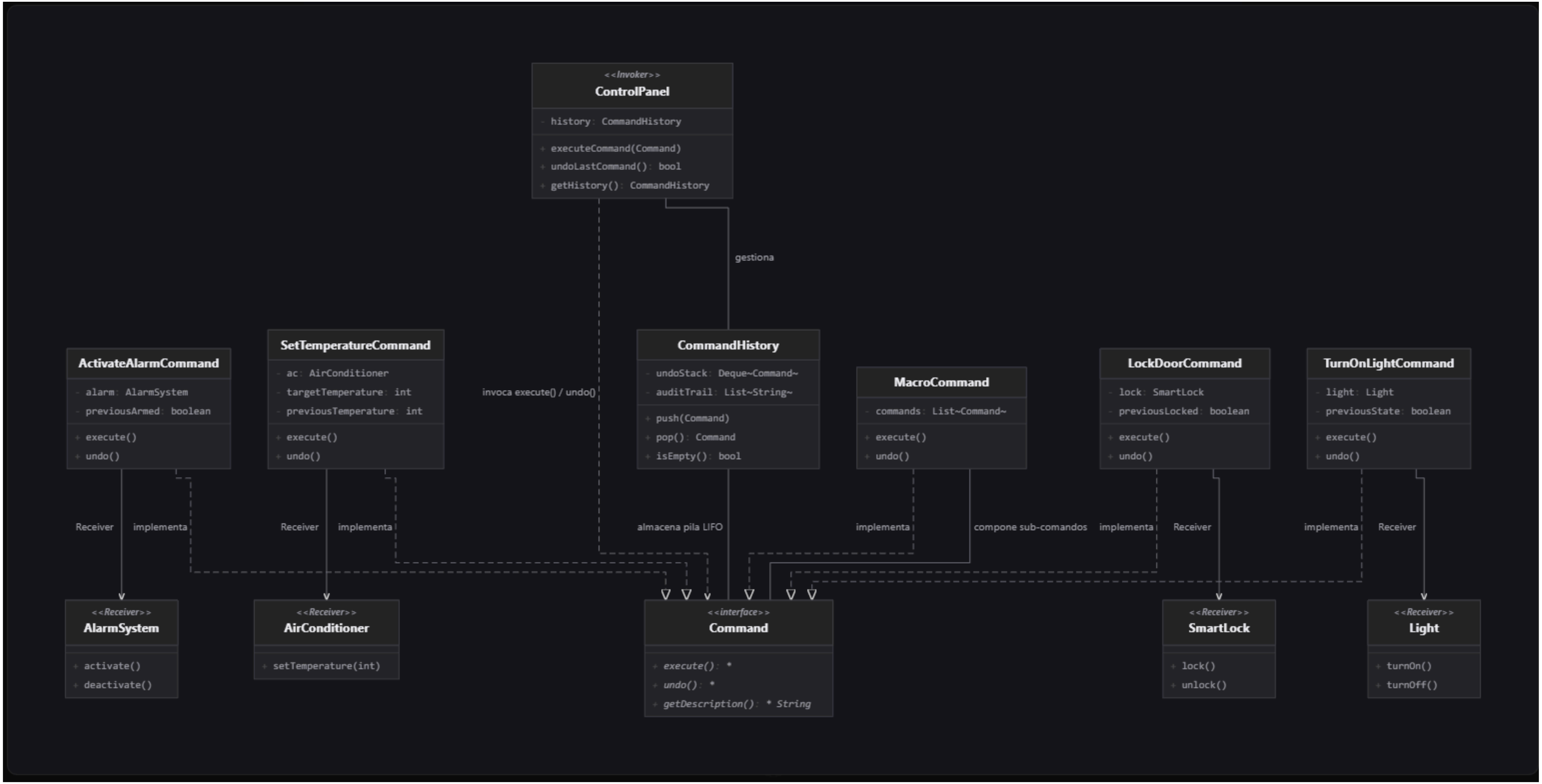
Caso práctico: sistema inteligente de automatización de edificios.

Software de automatización de edificios inteligentes que controla luces, climatización, cerraduras y alarmas. Su objetivo es demostrar el desacoplamiento arquitectónico mediante el patrón Command, permitiendo ejecutar acciones individuales, macros compuestas como el Modo Noche y la reversión completa de operaciones (Undo) mediante un historial de auditoría.



Arquitectura ANTES de aplicar Command

Arquitectura DESPUÉS de aplicar Command



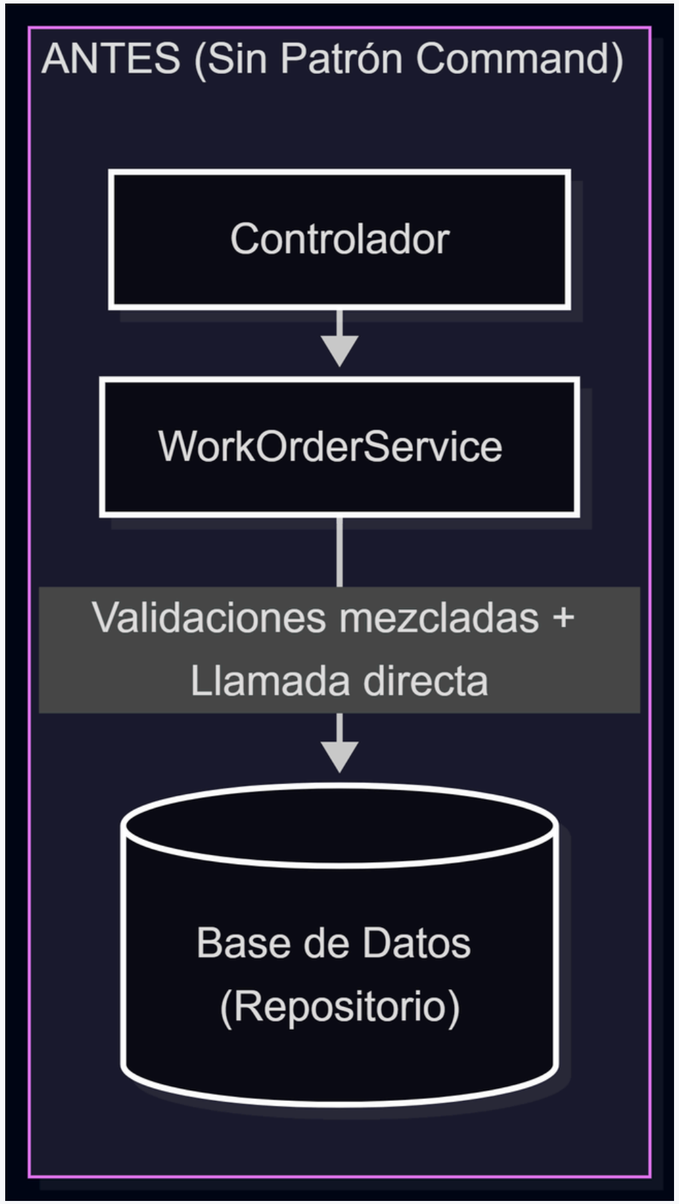
Caso práctico: TecnoFix sin Command

Limitaciones

Sobrecarga: el servicio valida y persiste todo.
Sin memoria: no se puede revertir.
Sin auditoría: no queda registro. Rigidez: cada función modifica el servicio.

Código TypeScript sin Command:

```
class WorkOrderService {
  async transitionStatus(id, to) {
    // validar transición y guardar
  }
  async assignTechnician(id, t) {
    // validar técnico y guardar
  }
}
```



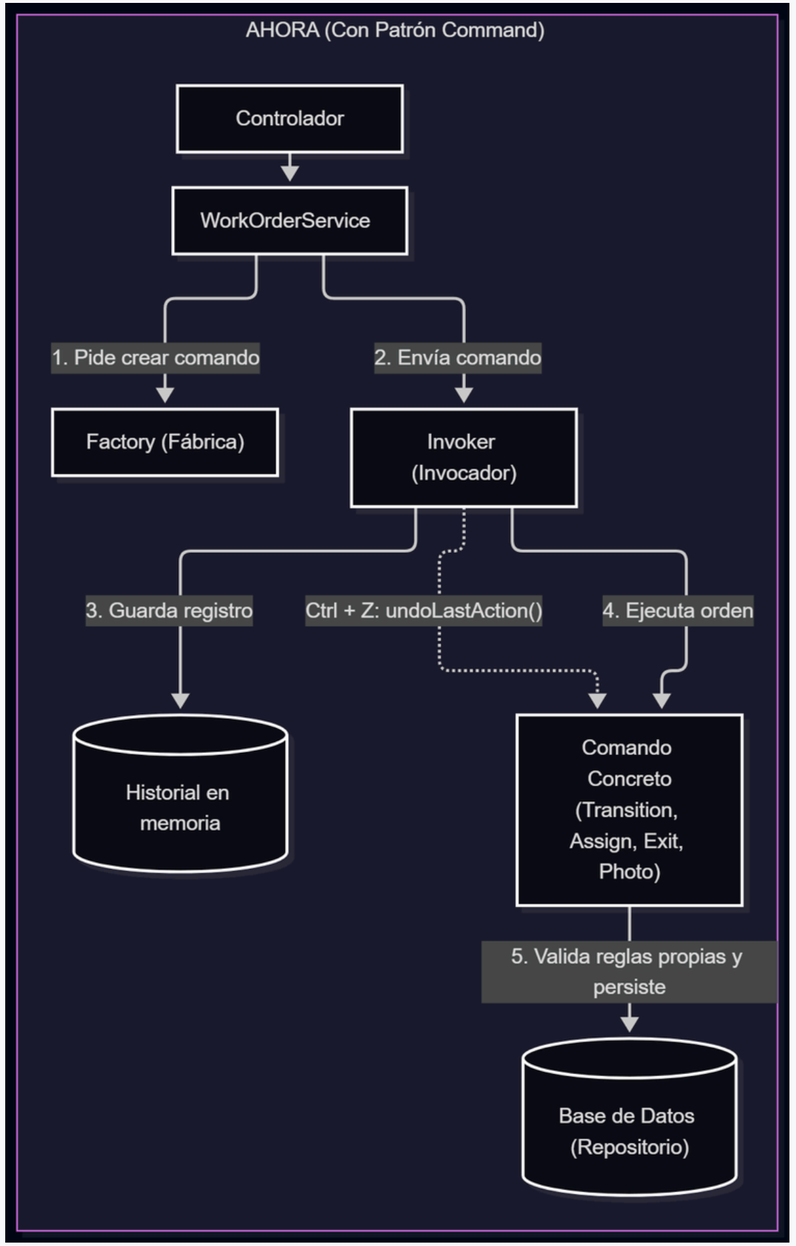
Lo que hace Command en TecnoFix

- **Separa solicitante y ejecutor:** se crea el comando y WorkOrderInvoker lo ejecuta.
- **Reglas propias:** máquina de estados, técnico ya asignado, entrega solo en LISTO_PARA_ENTREGA.
- **Deshacer:** TransitionStatusCommand guarda el estado previo y undoLastCommand() lo revierte.
- **Auditoría:** getHistory() devuelve descripción y fecha de cada comando.
- Extensión: un comando nuevo es una clase nueva.

Código con Command:

```
interface IWorkOrderCommand {
  execute(): Promise<WorkOrder>;
  getDescription(): string;
}
interface IUndoableCommand
  extends IWorkOrderCommand {
  undo(): Promise<WorkOrder>;
  canUndo(): boolean;
}
await invoker.executeCommand(cmd);
await invoker.undoLastCommand();
```

Diagrama de clases después del patrón



BALANCE CRÍTICO E INGENIERÍA

Ventajas, limitaciones y conclusiones

Ventajas

Desacoplamiento y operaciones independientes; nuevas acciones sin modificar invocadores.
Colas, historial y reversión, cuando se implemente y sea posible.

Limitaciones

Más clases y abstracción; puede ser excesivo para sistemas pequeños.
La complejidad también dificulta el seguimiento de la ejecución.

Conclusión: Command permite separar el objeto que solicita una operación del objeto que realmente la ejecuta.

Muchas gracias por su atención.

Repositorio del proyecto
Patron-command
URL:
https://github.com/David18Palomino/Patron-command.git